Reporte Técnico de Actividades Práctico-Experimentales Nro.6

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Alex Francisco Sigcho Serrano
Asignatura	Simulación
Ciclo	5to
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad..
Práctica Nro.	06
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	José O. Guamán Q.
Fecha	05 de noviembre de 2026 06 de noviembre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Evaluar la calidad estadística de secuencias de números pseudoaleatorios mediante las pruebas Chi-cuadrado y Kolmogorov-Smirnov, para determinar si son compatibles con una distribución uniforme en [0,1).

- Generar secuencias reproducibles con un generador congruencial lineal (LCG).
- Aplicar Chi-cuadrado a diez subintervalos y examinar las frecuencias observadas y esperadas.
- Calcular el estadístico D de Kolmogorov-Smirnov y su valor p frente a la uniforme U(0,1).
- Interpretar los valores p con nivel de significancia α = 0,05.
- Comparar configuraciones del generador y verificar la coincidencia entre Python y Java.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador.
- Lenguaje de programación Python.
- Lenguaje de programación Java.
- Entorno de desarrollo integrado.
- Generador de números pseudoaleatorios
- Python.
- NumPy.
- Matplotlib.
- Pandas.
- JDK.
- IntelliJ IDEA, Eclipse o NetBeans.
- Visual Studio Code

4. Procedimiento / Metodología Ejecutada

El proyecto se organizó en carpetas independientes para Python y Java, con una funcionalidad principal por módulo. El procedimiento ejecutado para N = 10 000 fue el siguiente:

1. Verificar Python y JDK, activar el entorno virtual existente con source venv/bin/activate y comprobar las dependencias.
2. Definir la semilla, el multiplicador a, el incremento c y el módulo m de cada configuración LCG.

3. Generar 10 000 valores $U_i = X_i/m$ con la recurrencia $X_{i+1} = (a \cdot X_i + c) \bmod m$ y conservar la secuencia.
4. Dividir [0,1) en k = 10 intervalos; calcular O_i, $E_i = N/k$ y $\chi^2 = \Sigma\,(O_i - E_i)^2/E_i$, con gl = k−1 = 9.
5. Ordenar los valores para KS y obtener $D^+ = \max(i/N - U(i))$, $D^- = \max(U(i) - (i-1)/N)$ y $D = \max(D^+, D^-)$.
6. Calcular los valores p y decidir si se rechaza H_0 al comparar con α = 0,05. Como ampliación teórica, ejecutar la prueba de rachas.
7. Ejecutar ambos lenguajes mediante ./ejecutar.sh 10000 y comparar frecuencias, estadísticos y los números generados; exportar CSV y gráficos.
8. Para la tabla complementaria solicitada por la guía, estimar π mediante pares (u, v), contabilizando los que satisfacen $u^2+v^2 \le 1$; calcular error absoluto $|\pi_{estimado} - \pi|$.

Parámetros de las configuraciones empleadas

Configuración	Semilla X_0	a	c	m
glibc	1	1 103 515 245	12 345	2^{32}
Numerical Recipes	42	1 664 525	1 013 904 223	2^{32}

Criterio de decisión: $p < 0{,}05 \rightarrow$ rechazar H_0; $p \ge 0{,}05 \rightarrow$ no rechazar H_0. No rechazar H_0 no demuestra que el generador sea verdaderamente aleatorio.

5. Resultados

La ejecución enviada desde el equipo Linux confirma el procesamiento de N = 10 000 valores por configuración, tanto en Python como en Java. Se presentan los resultados de dicho registro y las evidencias gráficas del proyecto.

5.1. Frecuencias obtenidas con Chi-cuadrado

Intervalo	glibc: O_i	N. Recipes: O_i	E_i (ambos)
[0.0, 0.1)	1001	1005	1000
[0.1, 0.2)	973	1011	1000
[0.2, 0.3)	989	964	1000
[0.3, 0.4)	1015	963	1000
[0.4, 0.5)	1005	1009	1000
[0.5, 0.6)	984	940	1000
[0.6, 0.7)	1017	1039	1000
[0.7, 0.8)	985	1013	1000
[0.8, 0.9)	1022	1030	1000
[0.9, 1.0)	1009	1026	1000

Con N = 10 000 y k = 10, la frecuencia esperada por intervalo es 1 000 ≥ 5; se cumple el requisito de aproximación de Chi-cuadrado.

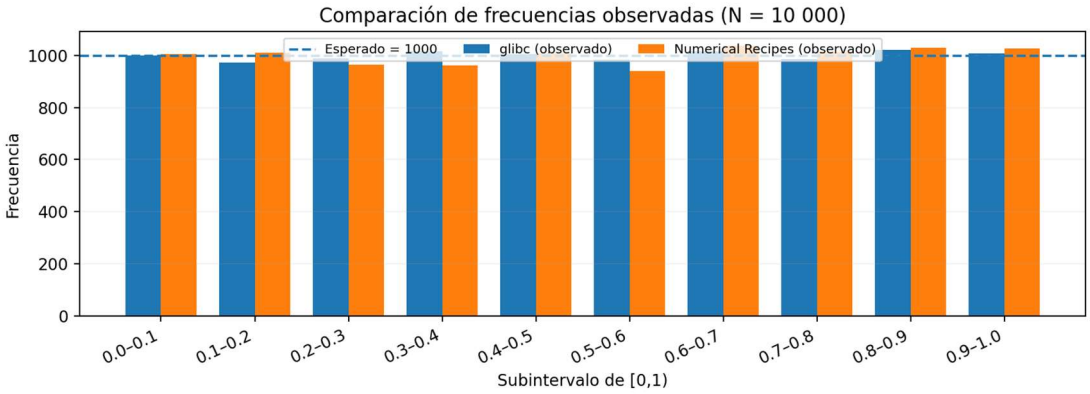


Figura 1. Distribución por intervalos de los 10 000 valores de cada configuración. Barras: frecuencias observadas; línea: frecuencia esperada.

5.2. Estadísticos, valores p y tiempos registrados

Generador / lenguaje	χ^2	p(χ^2)	D KS	p(KS)	p(rachas)	Tiempo (s)
glibc / Python	2,4360	0,982555	0,007134	0,687406	0,888654	0,023
glibc / Java	2,4360	0,982555	0,007134	0,687406	0,888654	0,032
N. Recipes / Python	9,7580	0,370429	0,012506	0,086936	0,617057	0,022
N. Recipes / Java	9,7580	0,370429	0,012506	0,086936	0,617057	0,017

Fuente: salida de terminal de la ejecución ./ejecutar.sh 10000 enviada por el estudiante. Los tiempos corresponden a una ejecución puntual y no permiten establecer superioridad permanente de un lenguaje.

Interpretación: en ambas configuraciones, los valores p de Chi-cuadrado, KS y rachas son mayores que 0,05. Por consiguiente, las pruebas aplicadas no rechazan sus respectivas hipótesis nulas para esta muestra. Chi-cuadrado y KS examinan uniformidad; rachas estudia determinados patrones de orden o independencia.

5.3. Verificación de la misma secuencia en ambos lenguajes

El comparador registró «VALIDADO» para glibc y Numerical Recipes y verificó 10 000 valores idénticos por configuración al comparar las secuencias de Python y Java (20 000 valores evaluados en total). Los resultados numéricos son equivalentes dentro de la tolerancia implementada de 1×10^{-5}.

5.4. Tabla requerida por la guía: π estimado, error y tiempo

La página 6 de la guía presenta una tabla para N = 1 000, 10 000, 100 000 y 1 000 000. Se reproduce aquí para la configuración glibc. Las estimaciones de π y sus errores se calcularon de manera reproducible a partir de los parámetros y del algoritmo del proyecto; solo el registro de N = 10 000 aporta tiempos medidos en el equipo del estudiante.

N	π estimado[1]	Error absoluto[1]	Tiempo Python	Tiempo Java
1 000	3,040000	0,101593	Por medir	Por medir
10 000	3,127200	0,014393	0,023 s	0,032 s
100 000	3,144160	0,002567	Por medir	Por medir
1 000 000	3,142648	0,001055	Por medir	Por medir

Cálculos reproducibles a partir del código LCG; los valores para N ≠ 10 000 no son evidencia de una ejecución local del estudiante. Para obtener los tiempos faltantes, ejecutar ./ejecutar.sh sin argumentos y trasladar los valores medidos de la tabla CSV generada.

Para N = 10 000, la configuración Numerical Recipes produjo π estimado = 3,112800 y error absoluto = 0,028793. Sus tiempos registrados fueron 0,022 s en Python y 0,017 s en Java. La estimación de π es una extensión complementaria de la tabla de la guía, no sustituye las pruebas de uniformidad.

5.5. Evidencias gráficas de Kolmogorov-Smirnov

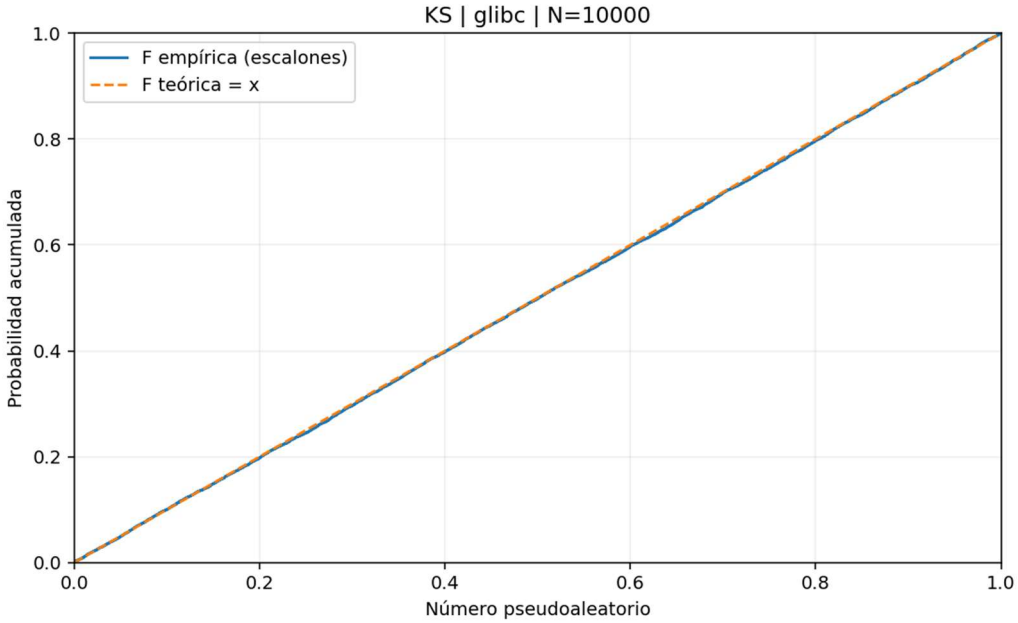


Figura 2. Curvas acumuladas teórica y empírica para glibc, N = 10 000 (gráfico incluido en los resultados del proyecto).

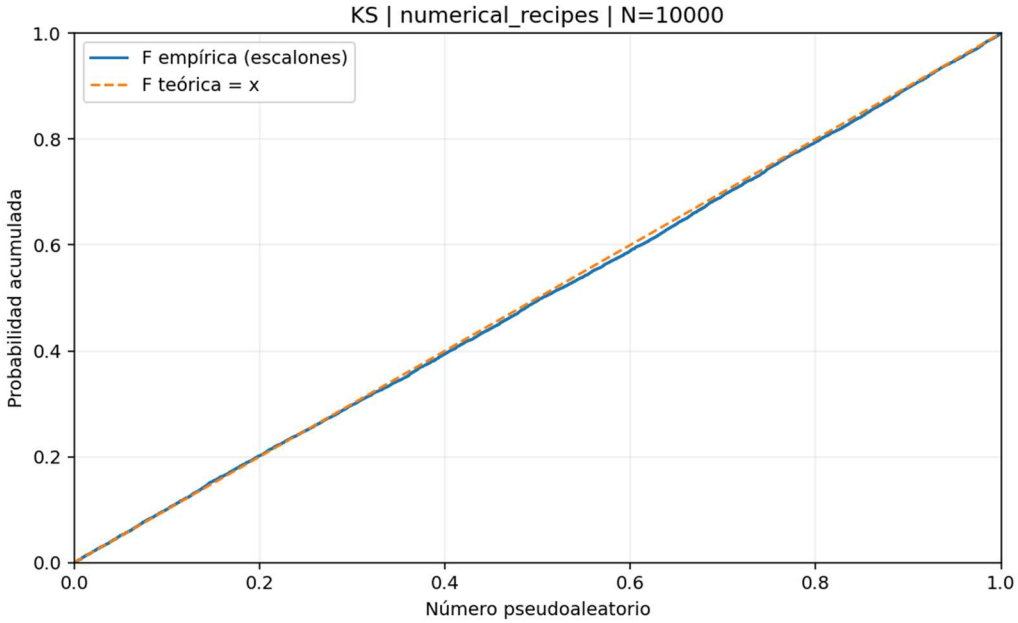


Figura 3. Curvas acumuladas teórica y empírica para Numerical Recipes, N = 10 000 (gráfico incluido en los resultados del proyecto).

6. Preguntas de Control

1. ¿Qué es la hipótesis nula?

Es la afirmación inicial que se contrasta con los datos. Para Chi-cuadrado y KS, H_0 plantea que los valores son compatibles con una distribución uniforme U(0,1). En la prueba de rachas, H_0 corresponde a un orden compatible con independencia, según el tipo de contraste empleado.

2. ¿Qué representa el valor p?

Es la probabilidad, suponiendo verdadera H_0 y las condiciones del modelo estadístico, de obtener un estadístico al menos tan extremo como el observado. No representa la probabilidad de que H_0 sea verdadera [1].

3. ¿Qué significa rechazar H_0?

Significa que la evidencia observada resulta estadísticamente incompatible con H_0 al nivel de significancia fijado; en esta práctica, $p < 0,05$. No implica identificar automáticamente la causa de la desviación.

4. ¿Qué diferencia existe entre Chi-cuadrado y KS?

Chi-cuadrado contrasta las frecuencias observadas y esperadas en intervalos discretizados; KS compara la función de distribución acumulada empírica con la función teórica continua y utiliza la máxima distancia entre ambas.

5. ¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?

Porque cada prueba examina características limitadas de una secuencia finita y puede no detectar otros patrones, correlaciones o defectos. La ausencia de rechazo no prueba la aleatoriedad; conviene combinar pruebas de uniformidad e independencia y, cuando corresponda, baterías más amplias [2].

7. Conclusiones

- La implementación del LCG generó 10 000 valores por configuración y permitió reproducir las mismas secuencias en Python y Java; el comparador confirmó coincidencia completa de los 20 000 valores evaluados.
- Las frecuencias observadas en diez intervalos se aproximaron a las 1 000 observaciones esperadas. Para glibc se obtuvo $\chi^2 = 2,4360$, y para Numerical Recipes $\chi^2 = 9,7580$, ambos con valores p superiores a 0,05.
- La prueba KS produjo $D = 0,007134$ para glibc y $D = 0,012506$ para Numerical Recipes. Sus valores p tampoco motivaron el rechazo de la uniformidad con $\alpha = 0,05$.
- La prueba adicional de rachas no detectó desviaciones significativas de los patrones de orden analizados. Sin embargo, superar estas pruebas no es prueba definitiva de independencia o de aleatoriedad verdadera.
- La comparación entre Python y Java evidencia equivalencia de los algoritmos, aunque los tiempos medidos presentaron pequeñas diferencias. La estimación de π sirve como aplicación complementaria de Monte Carlo.

8. Recomendaciones

- Ejecutar el programa sin argumentos para registrar N = 1 000, 10 000, 100 000 y 1 000 000; completar los tiempos de la tabla con resultados del propio equipo.
- Repetir la medición de tiempos varias veces y comunicar valores promedio o medianas, separando cálculos de tareas de exportación y gráficos.
- Mantener fijos semilla y parámetros en ambos lenguajes para asegurar reproducibilidad; documentar cualquier cambio de configuración.
- Complementar las pruebas con análisis seriales, póker u otras baterías cuando se requiera mayor confianza en un generador de simulación.
- Incluir capturas de terminal, archivos CSV y las imágenes creadas por las ejecuciones finales como anexos verificables.

9. Reflexión crítica

Esta práctica permitió comprender que generar números pseudoaleatorios no garantiza por sí solo su idoneidad para una simulación. Aprendí a distinguir la uniformidad de la independencia y a aplicar Chi-cuadrado y KS para evaluar cómo se distribuyen los valores, además de rachas para examinar el orden de la secuencia. También pude interpretar correctamente el valor p: no rechazar una hipótesis no equivale a demostrar que es verdadera. La implementación paralela en Python y Java reforzó la importancia de definir algoritmos y parámetros de forma exacta para reproducir resultados; finalmente, la comparación de tiempos evidenció que las mediciones de

rendimiento dependen de las condiciones de ejecución y requieren repetición para extraer conclusiones generales.

10. Bibliografía / Referencias

[1] R. L. Wasserstein y N. A. Lazar, «The ASA Statement on p-Values: Context, Process, and Purpose», The American Statistician, vol. 70, n.º 2, pp. 129–133, 2016. DOI: 10.1080/00031305.2016.1154108

[2] P. L'Ecuyer y R. Simard, «TestU01: A C Library for Empirical Testing of Random Number Generators», ACM Transactions on Mathematical Software, vol. 33, n.º 4, art. 22, 2007. DOI: 10.1145/1268776.1268777